Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

la

Lucrarea de laborator Nr. 4

Disciplina: Tehnici de programare

Tema:

A efectuat : st. gr.SI –264 Pavalachi Veronica

A verificat : asistent universitar – Reitman Patricia

Chișinău – 2026

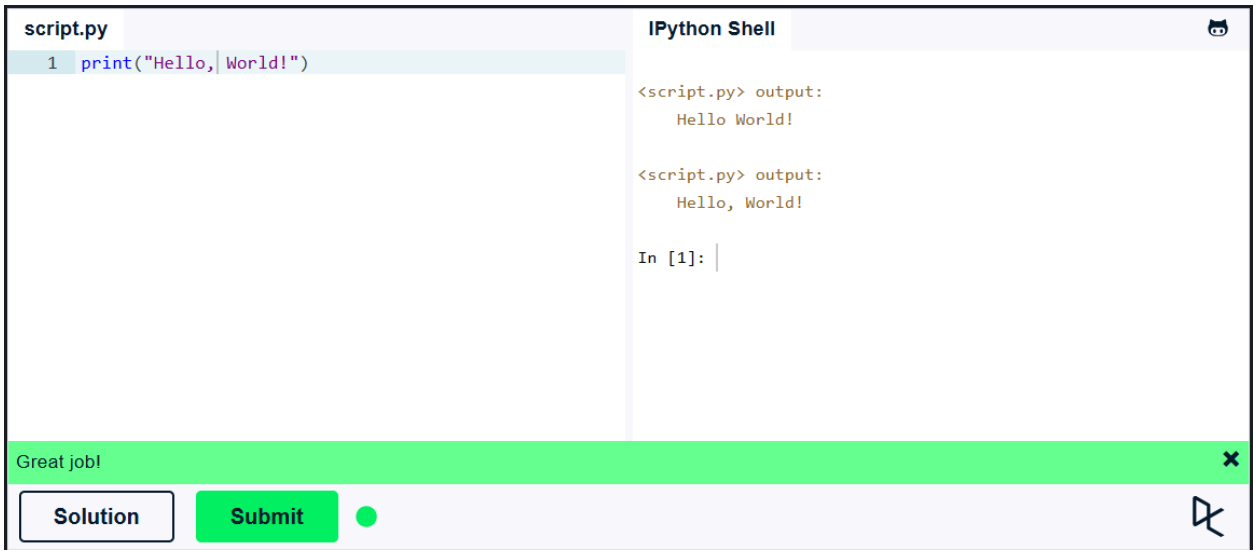


Figura 1-Functia print()

În acest exercițiu am folosit funcția print() pentru a afișa un mesaj pe ecran. Am scris textul „Hello, World!” între ghilimele. După rulare, mesajul a apărut în partea dreaptă.

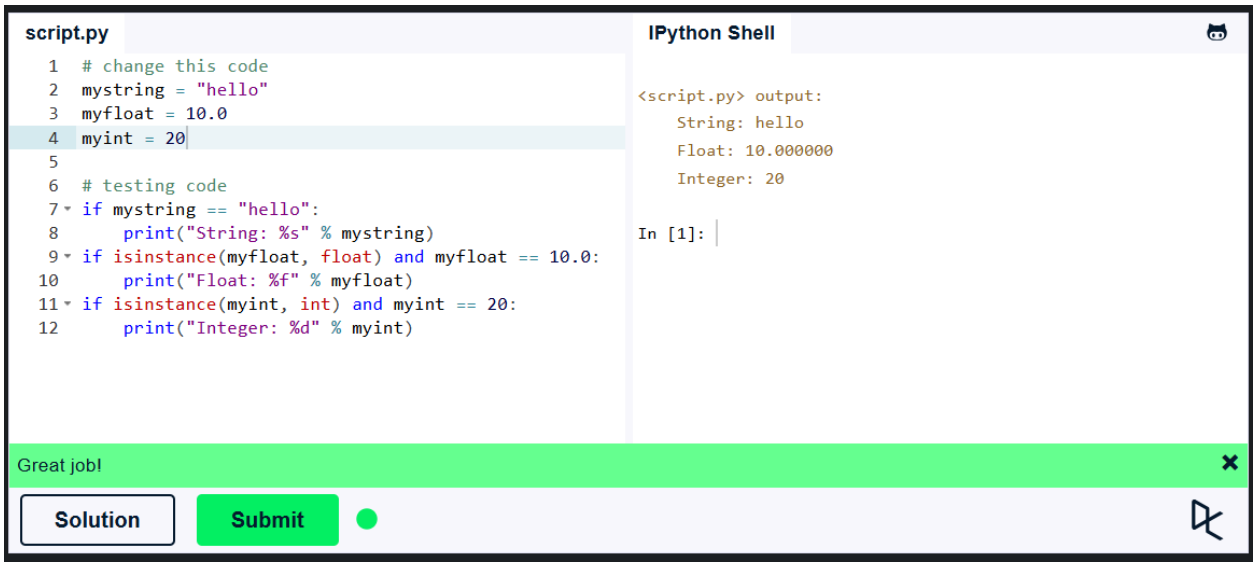


Figura 2-Variabile si Tipuri

Am creat trei variabile: mystring pentru textul „hello”, myfloat pentru numărul cu zecimale 10.0 și myint pentru numărul întreg 20. Variabilele păstrează aceste valori. La final, programul le-a afișat pe ecran.

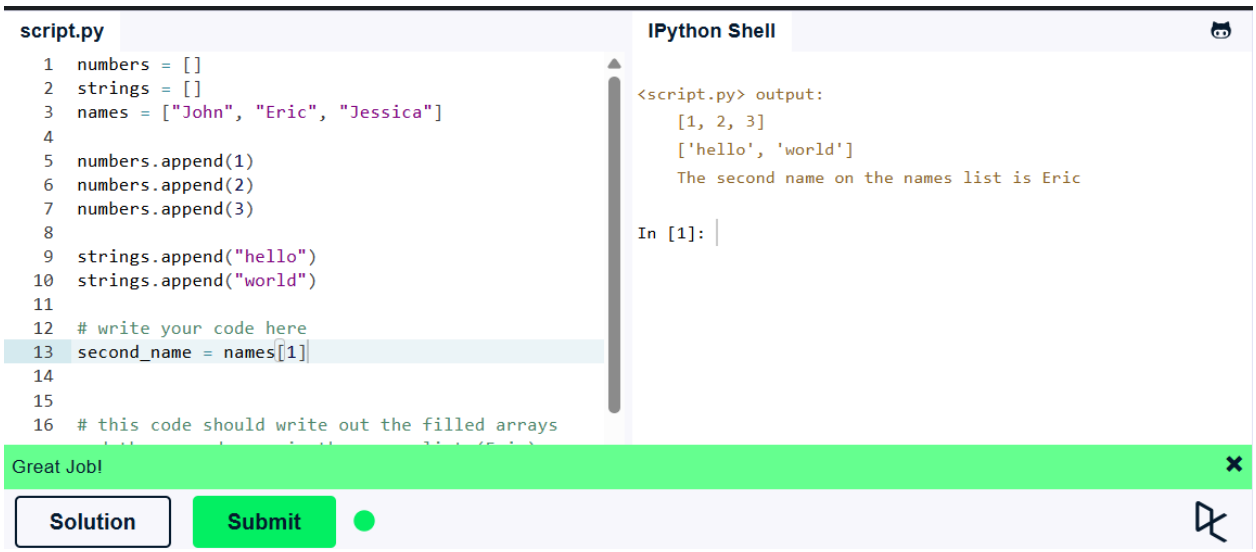


Figura 3-Liste

Am lucrat cu liste, în care putem păstra mai multe valori. Cu append() am adăugat numerele 1, 2, 3 și cuvintele „hello” și „world”. Cu names[1] am ales al doilea nume, „Eric”, deoarece numărarea pozițiilor începe de la 0.

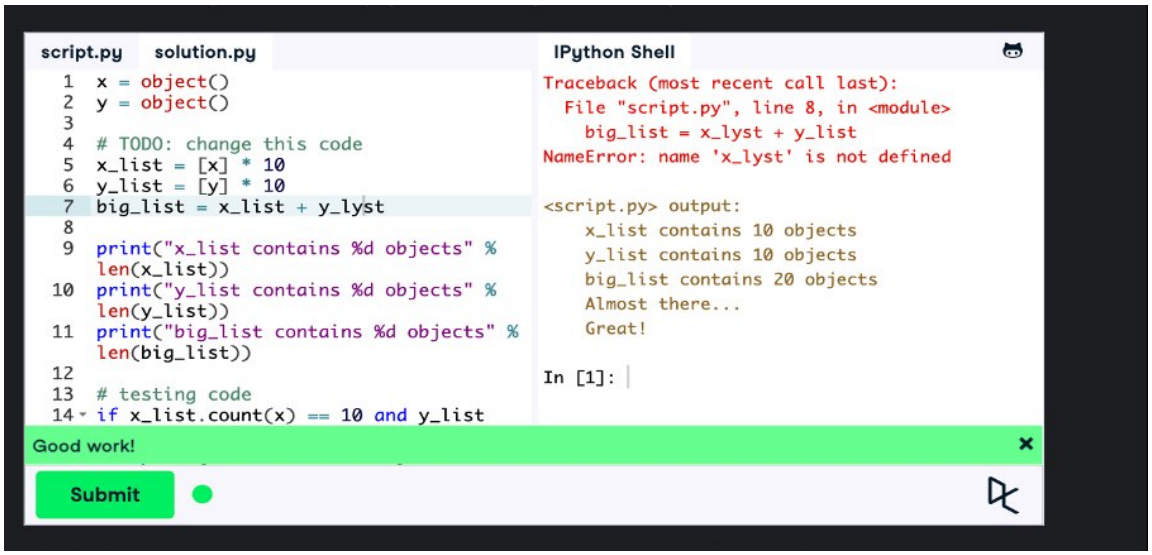


Figura 4-Operatori Basic

Am creat două liste, fiecare cu câte 10 elemente, folosind * 10. Cu semnul + am unit listele într-o singură listă, cu 20 de elemente. Cu len() am aflat câte elemente are fiecare listă.

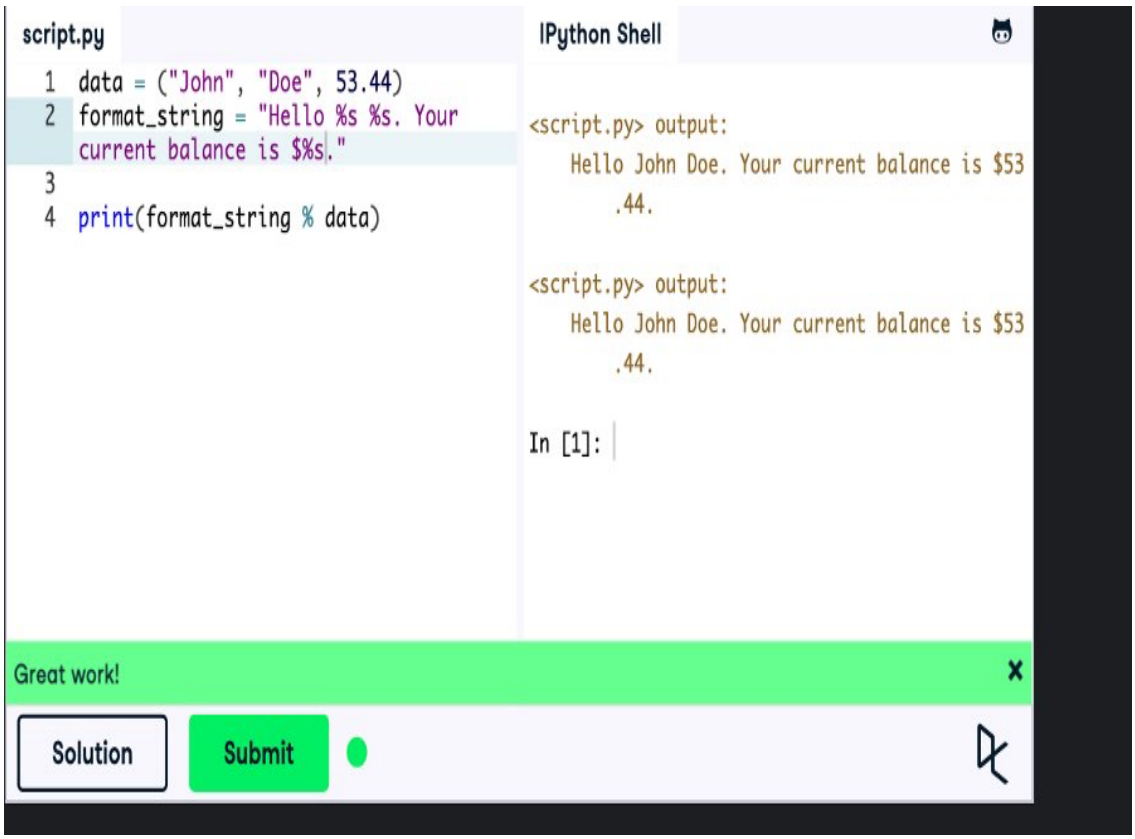


Figura 5-Formatarea Stringurilor

Am afișat numele „John Doe” și suma 53.44 într-un mesaj. Am folosit %s pentru a pune valorile în locurile dorite din text. La final, print() a afișat mesajul complet.

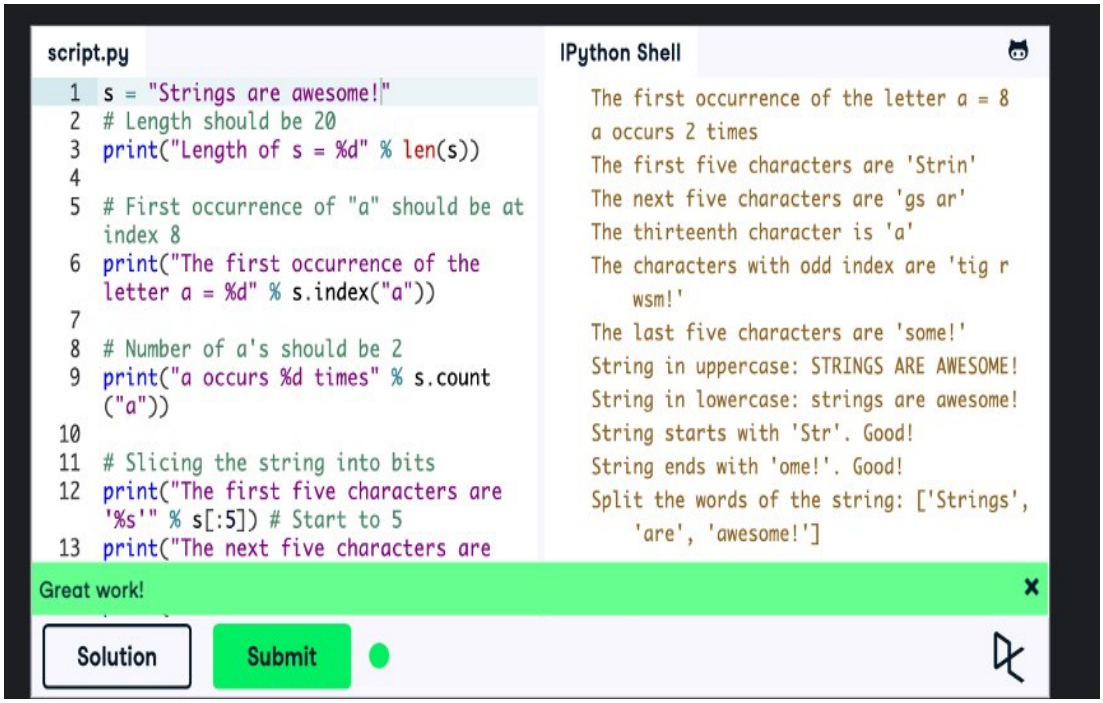


Figura 6-Operatori Basic String

Am lucrat cu textul „Strings are awesome!”. Cu len() am aflat lungimea, cu index() poziția primei litere „a”, iar cu count() de câte ori apare. Am ales și bucăți din text. Cu upper() și lower() am schimbat literele în mari și mici. Cu startswith() și endswith() am verificat începutul și sfârșitul, iar cu split() am separat cuvintele.

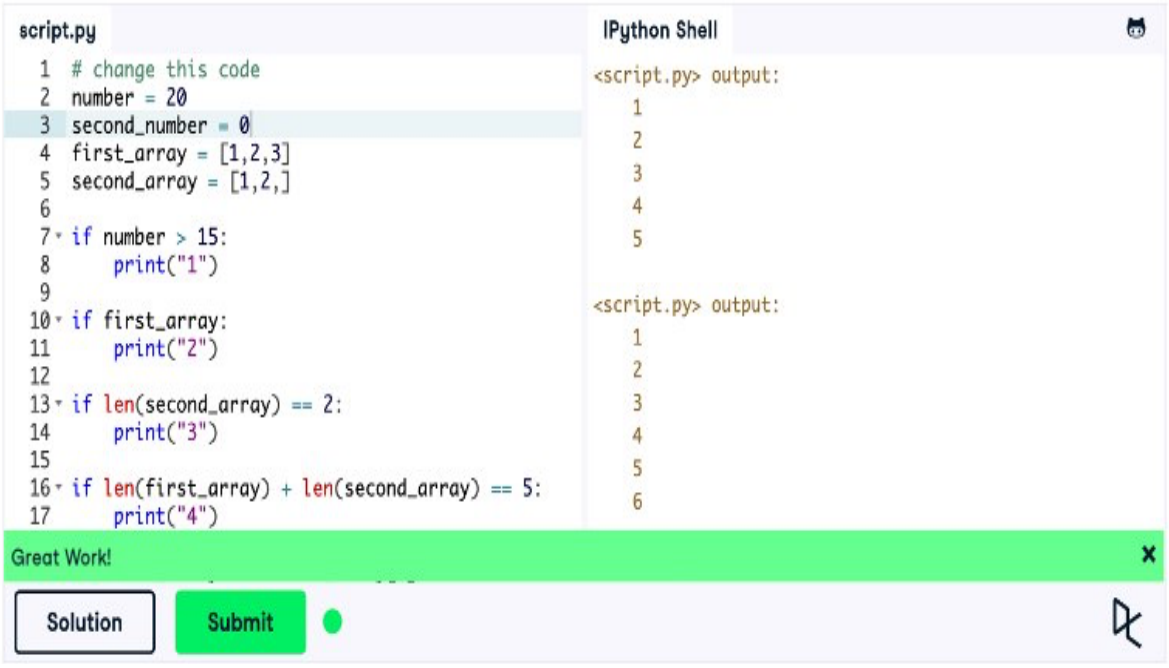


Figura 7-Conditii

Am schimbat valorile variabilelor pentru ca toate condițiile if să fie adevărate. Am pus number = 20, second_number = 0 și am completat cele două liste. Programul a verificat condițiile și a afișat numerele de la 1 la 6.

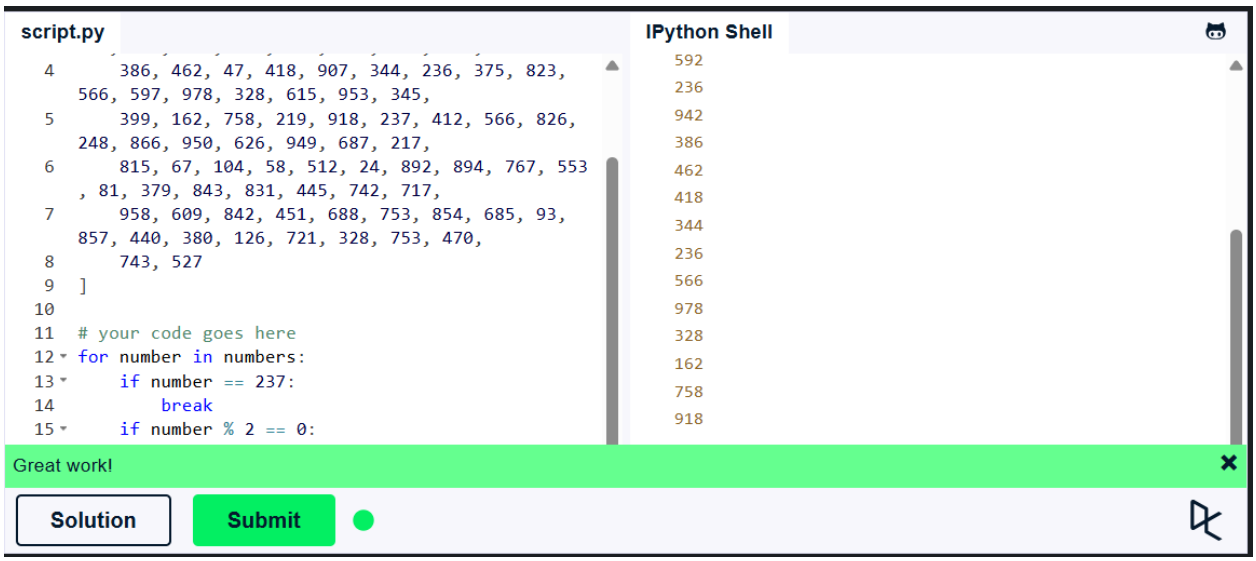


Figura 8-Loops

Am folosit for pentru a verifica pe rând numerele din listă. Condiția number % 2 == 0 verifică dacă un număr este par. Programul a afișat numerele pare și s-a oprit când a ajuns la 237, folosind break.

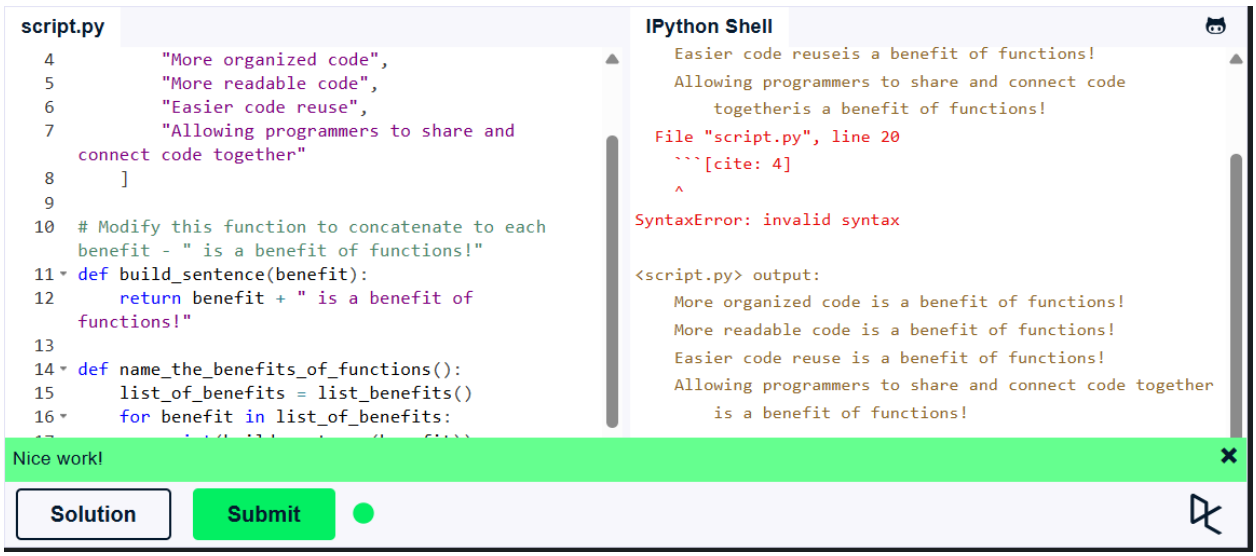


Figura 9-Functii

Am completat funcția build_sentence(), care primește un text și adaugă la el „is a benefit of functions!”. Cu return am trimis înapoi propoziția completă. La final, programul a afișat câte o propoziție pentru fiecare avantaj din listă.

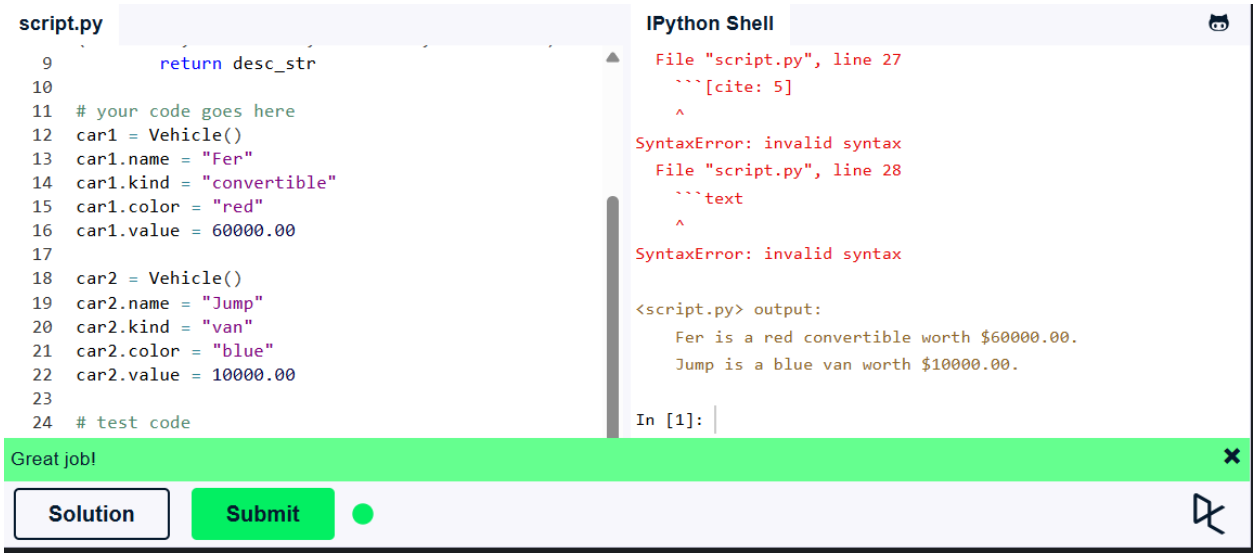


Figura 10-Clase si Obiecte

Am creat două obiecte din clasa Vehicle, adică două mașini după același model. Pentru fiecare am scris numele, tipul, culoarea și valoarea. La final, programul a afișat descrierea fiecărei mașini.

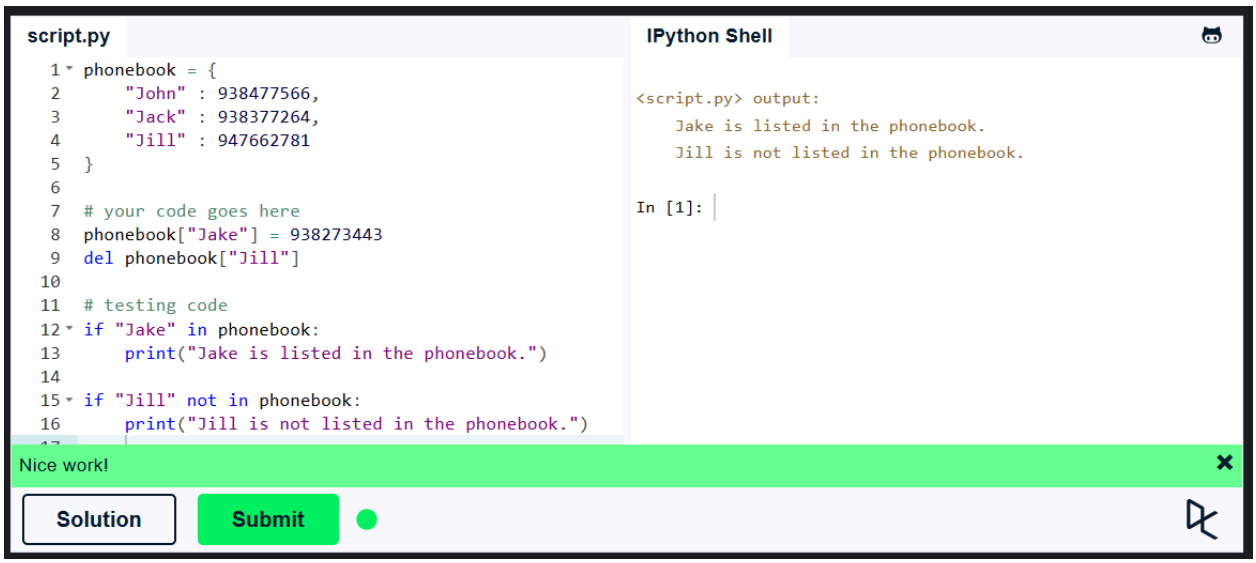


Figura 11-Dictionaries

Am lucrat cu un dicționar care păstrează nume și numere de telefon. Am adăugat numele „Jake” cu numărul lui și am șters „Jill” folosind del. Programul a confirmat că Jake se află în dicționar, iar Jill nu mai este acolo.

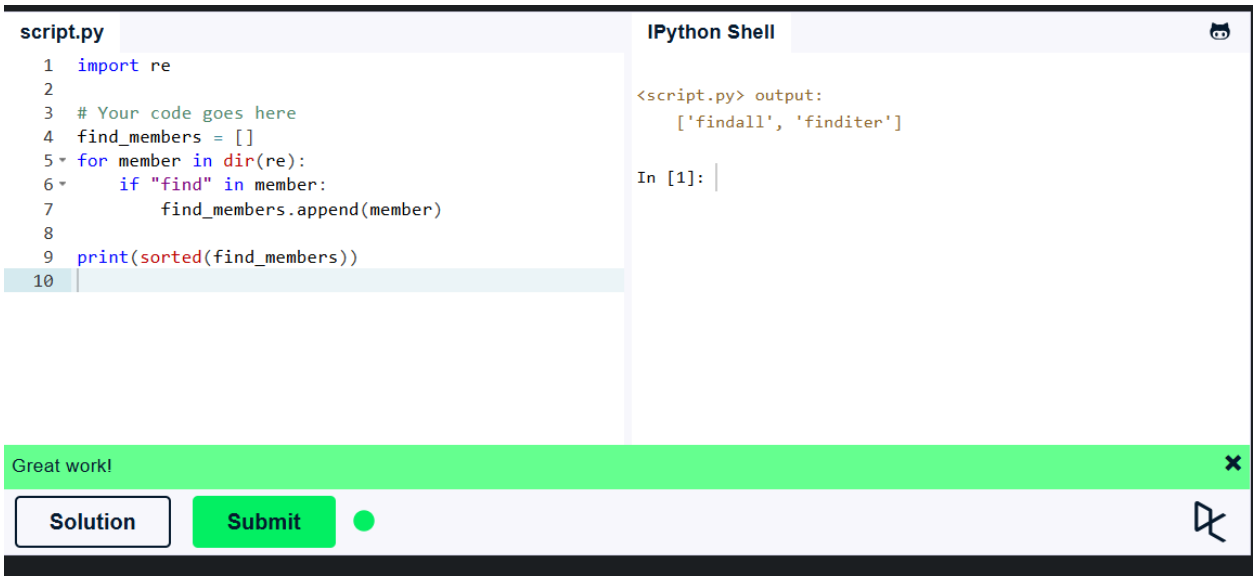


Figura 12- Modules and Packages

Am folosit import re pentru a putea lucra cu modulul re. Cu dir(re) am obținut numele disponibile în el și le-am ales pe cele care conțin „find”. Cu sorted() le-am pus în ordine alfabetică. Pe ecran au apărut „findall” și „finditer”.

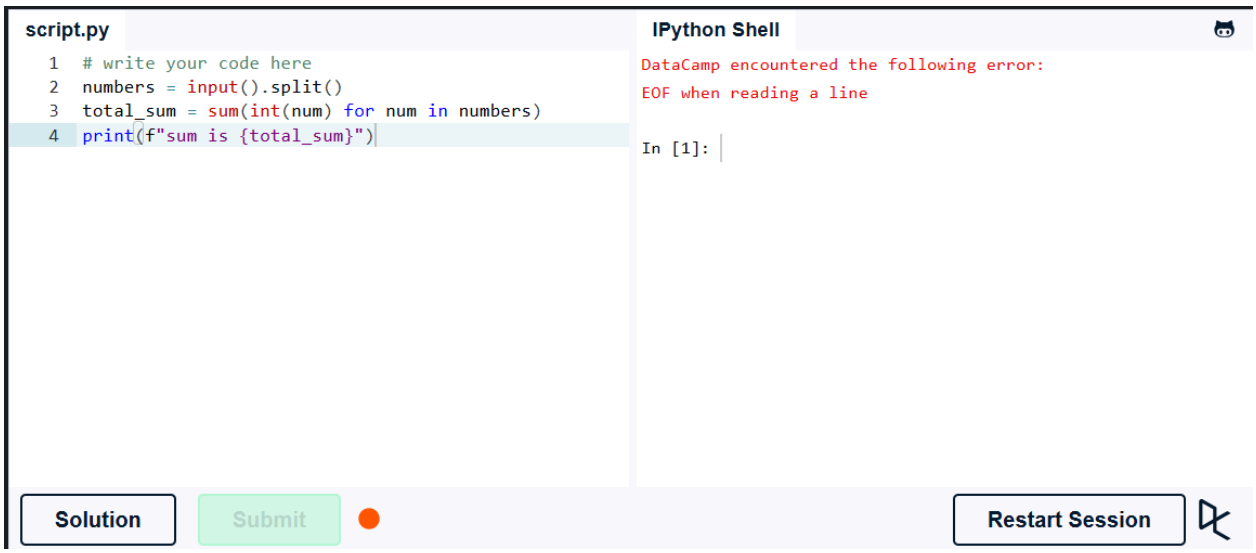


Figura 13-Input and Output

Am scris un program care trebuie să citească numere cu input() și să le separe cu split(). Cu int() le transformă în numere întregi, iar cu sum() le adună. În această rulare, programul nu a primit date pentru citire și a apărut eroarea „EOF when reading a line”. De aceea, suma nu a fost afișată.

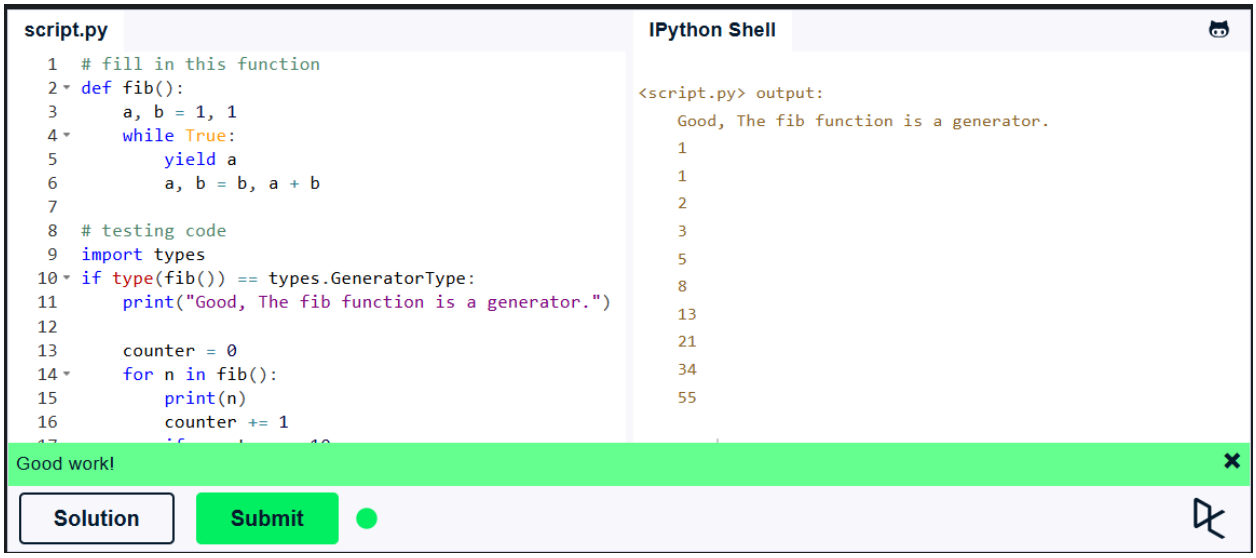


Figura 14-Generators

Am creat funcția fib() pentru a produce numerele Fibonacci. Fiecare număr nou este suma celor două numere dinainte. Cu yield, funcția oferă câte un număr pe rând. Programul a afișat primele 10 numere: 1, 1, 2, 3, 5, 8, 13, 21, 34 și 55.

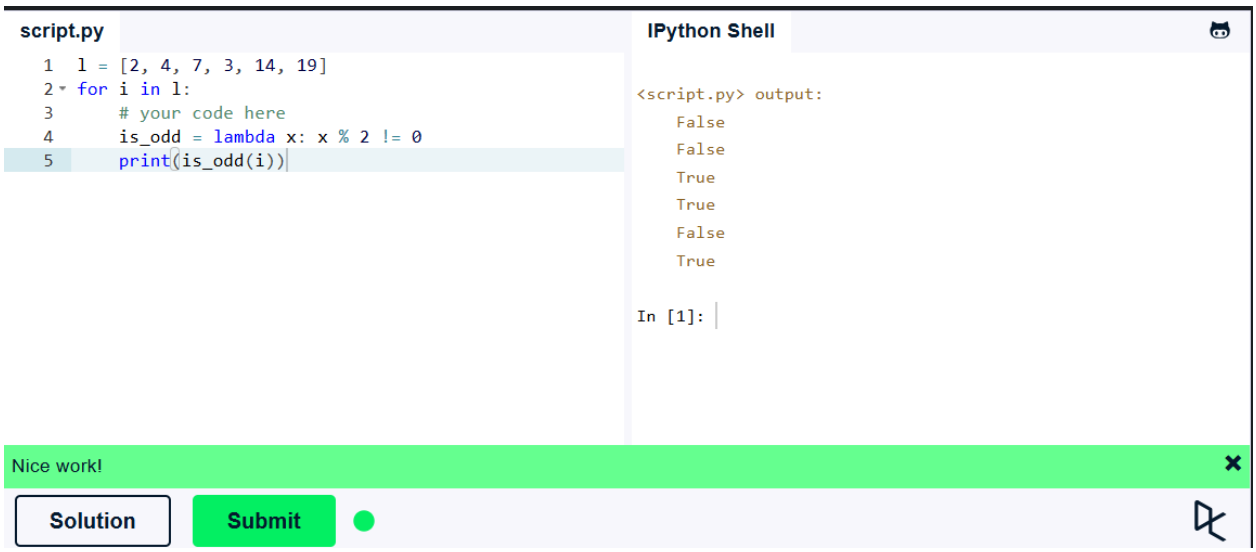


Figura 15-Lambda Functions

Am folosit lambda pentru a scrie o funcție scurtă care verifică dacă un număr este impar. Am verificat fiecare număr din listă. Programul a afișat True pentru numerele impare și False pentru cele pare.

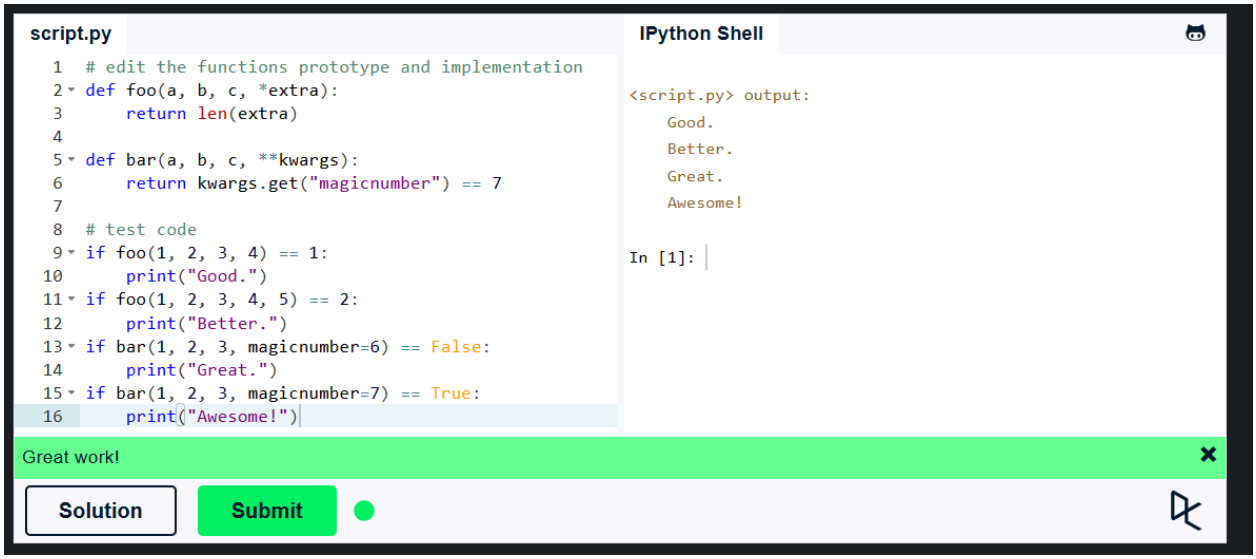


Figura 16-Multiple Function Arguments

Am creat funcții care pot primi mai multe valori. În foo(), *extra adună valorile suplimentare, iar len() le numără. În bar(), **kwargs primește valori scrise cu nume și verifică dacă magicnumber este 7. Mesajele afișate arată că verificările au reușit.

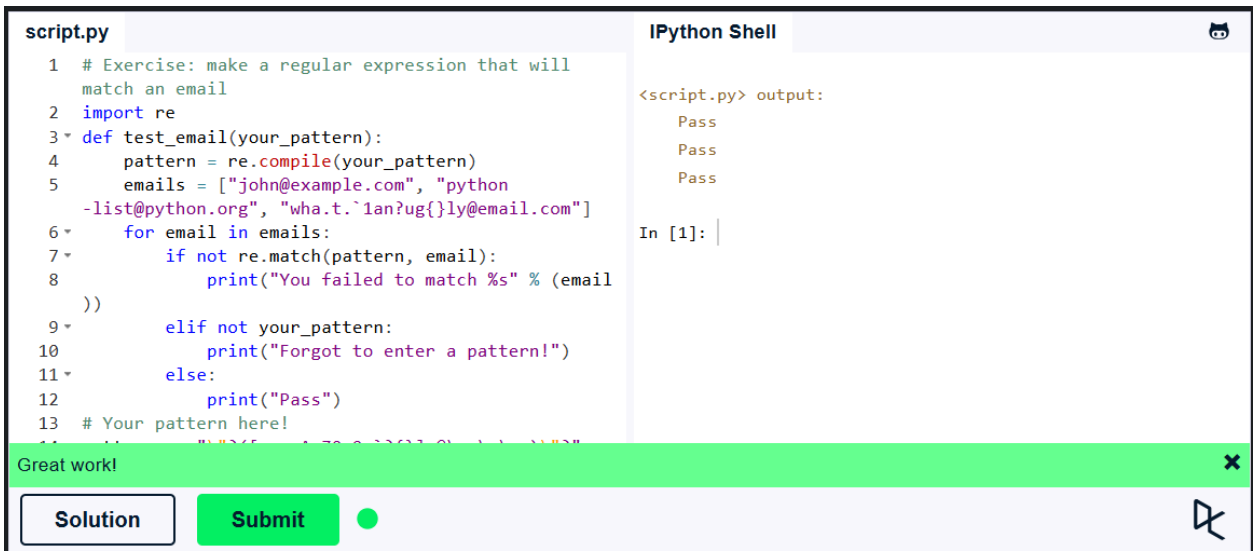


Figura 17-Regular Expressions

Am folosit modulul re pentru a căuta într-un text după un model. În acest exercițiu, modelul a fost folosit pentru adrese de e-mail. Programul a verificat cele trei adrese din exemplu și a afișat „Pass” pentru fiecare.

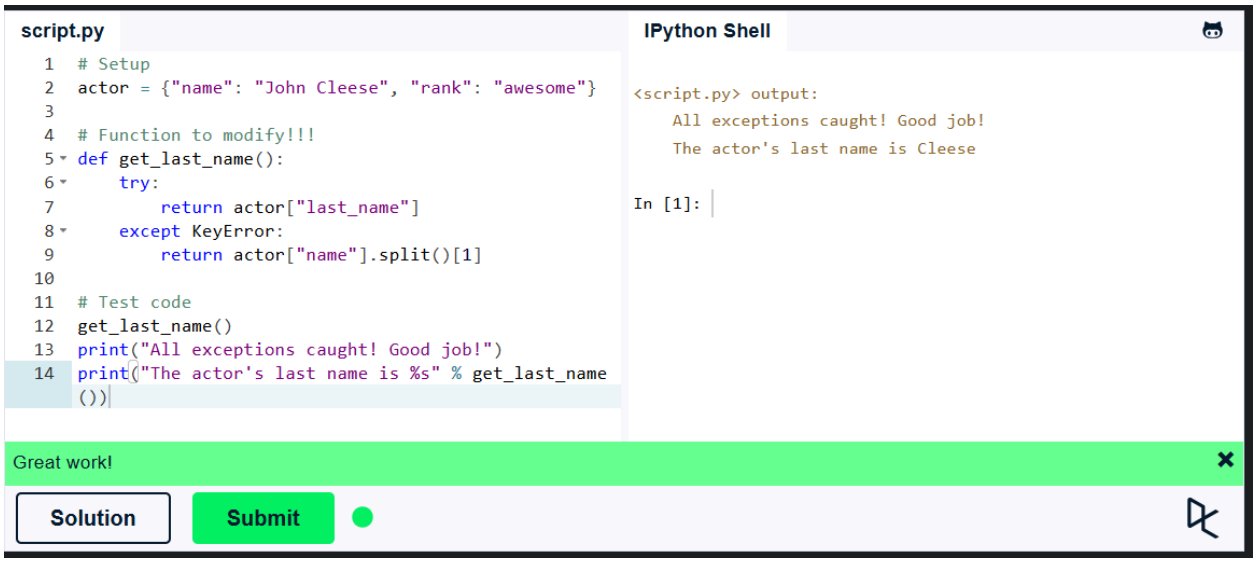


Figura 18-Exception Handeling

Am folosit try și except pentru a trata o eroare fără să oprim programul. În dicționar lipsea last_name. Programul a luat atunci textul „John Cleese”, l-a separat cu split() și a ales al doilea cuvânt. Pe ecran a apărut numele de familie „Cleese”.

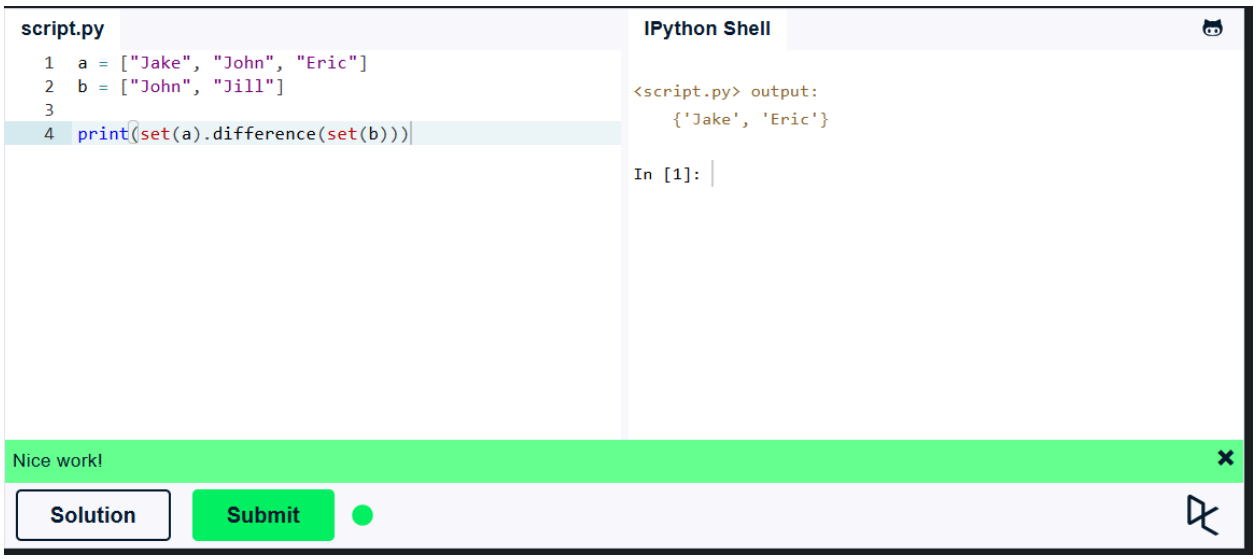


Figura 19-Sets

Am transformat cele două liste în mulțimi folosind set(). O mulțime păstrează fiecare valoare o singură dată. Cu difference() am ales numele care sunt în prima mulțime, dar nu sunt în a doua. Rezultatul conține „Jake” și „Eric”.

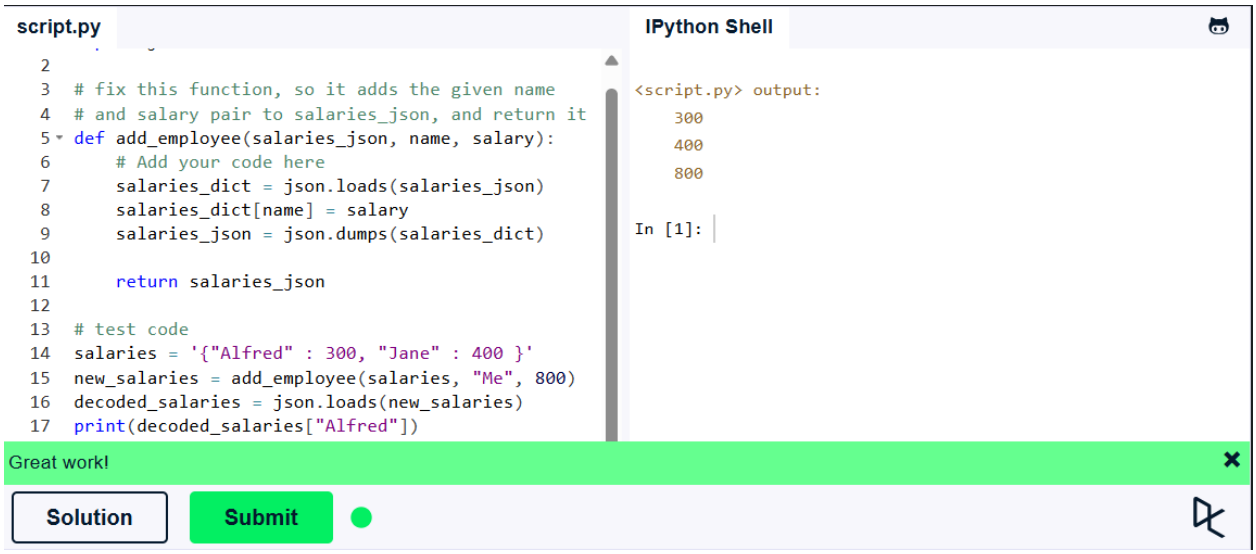


Figura 20-Serialization

Am lucrat cu date scrise în format JSON. Cu json.loads() le-am transformat într-un dicționar și am adăugat numele „Me” cu salariul 800. Cu json.dumps() le-am transformat înapoi în text JSON. La final, programul a afișat salariile 300, 400 și 800.

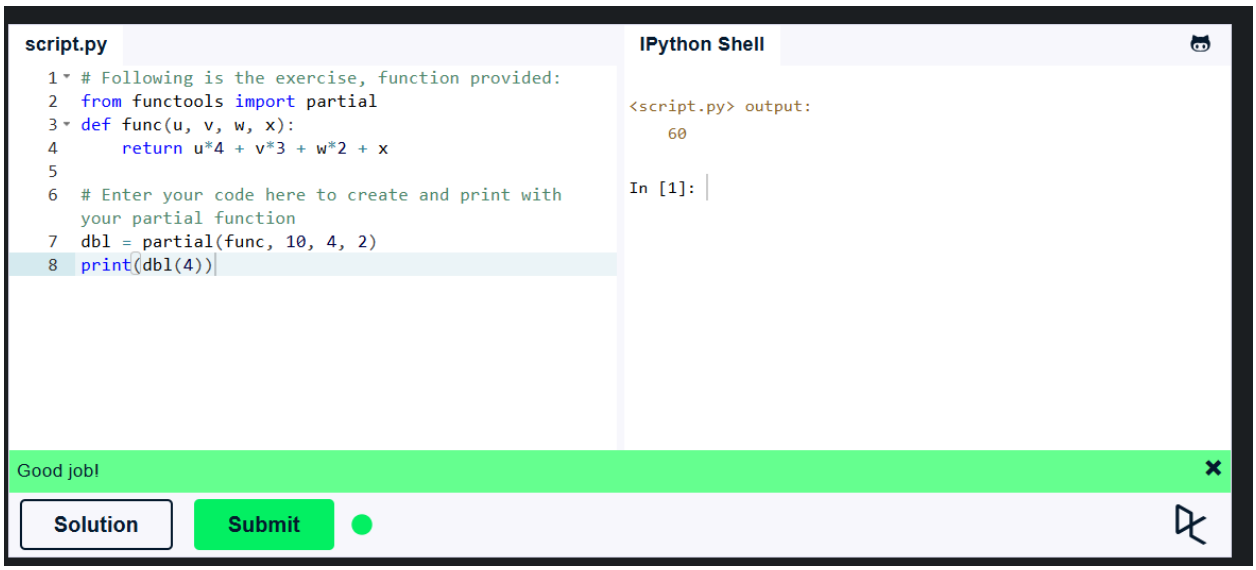


Figura 21-Partial Functions

Am folosit partial() pentru a fixa primele trei valori ale funcției: 10, 4 și 2. La apelul dbl(4), am dat doar ultima valoare. Programul a calculat $10 \times 4 + 4 \times 3 + 2 \times 2 + 4$ și a afișat rezultatul 60.

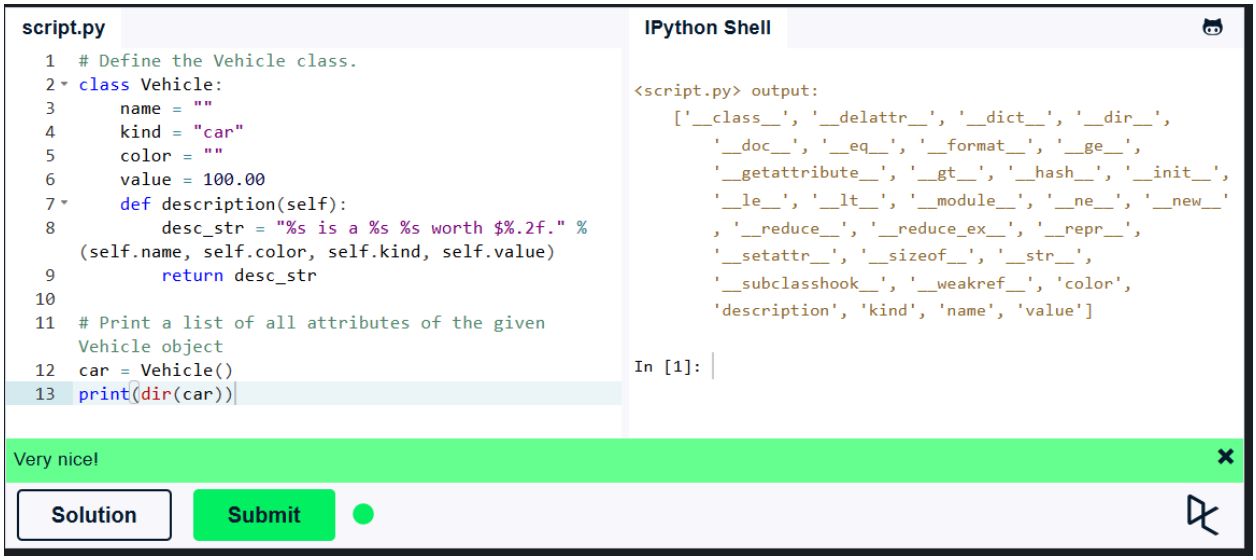


Figura 22-Code Introspection

Am creat obiectul car din clasa Vehicle. Cu dir(car) am afișat numele proprietăților și funcțiilor disponibile pentru acest obiect. În listă apar, de exemplu, name, kind, color, value și description.

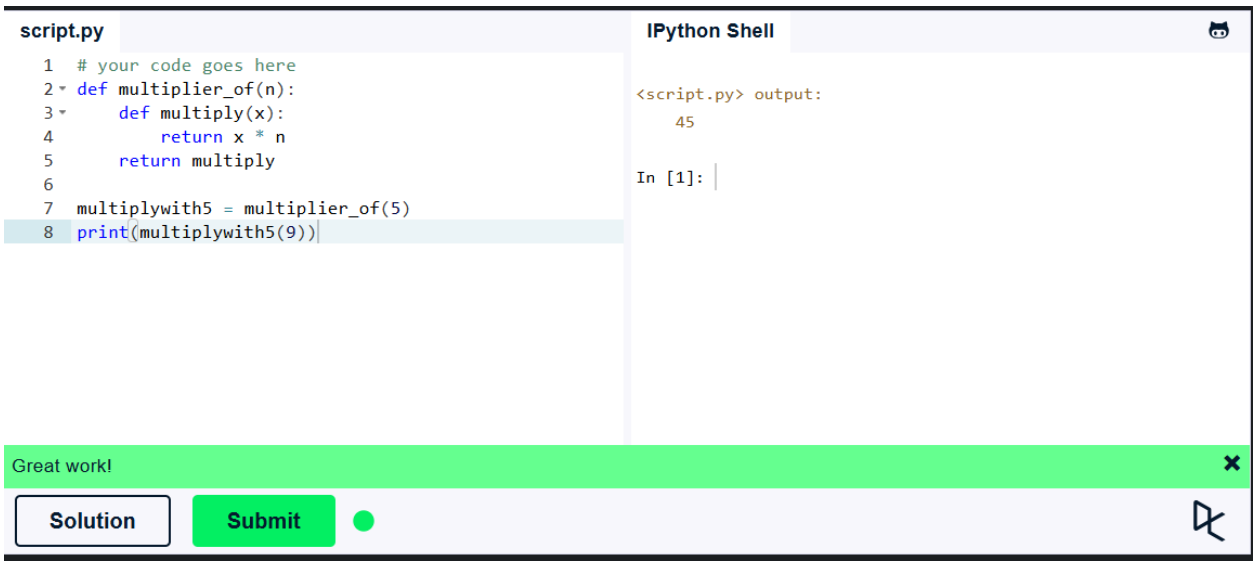


Figura 23-Closures

Am scris o funcție în interiorul altei funcții. Funcția din interior păstrează valoarea n și o folosește la înmulțire. Cu multiplier_of(5) am creat o funcție care înmulțește cu 5. Când i-am dat numărul 9, rezultatul a fost 45.

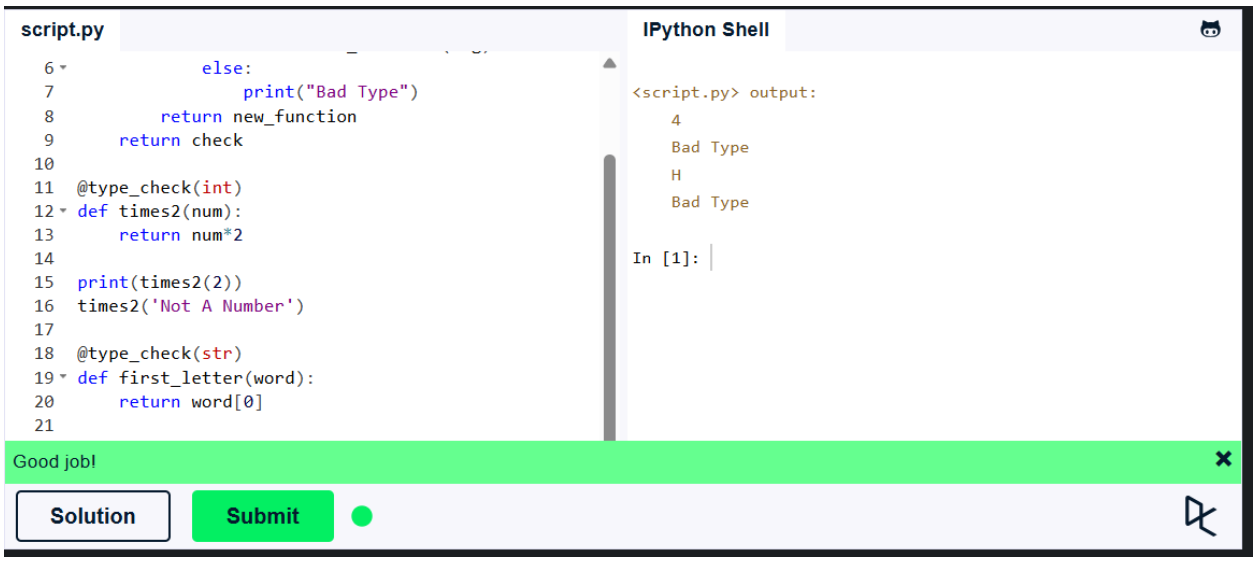


Figura 24-Decorators

Am folosit @type_check pentru a verifica tipul valorii primite de o funcție. Pentru times2() este acceptat un număr întreg, iar pentru first_letter() este acceptat un text. Programul a afișat 4 și litera „H” pentru valorile potrivite. Pentru celelalte a afișat „Bad Type”.